Architecture et rapport d'implémentation d'un plugin LeRobot : Modèles de monde V-JEPA 2.1 et Model Predictive Control pour le bras robotique SO-101

La robotique moderne traverse une phase de transition majeure, s'éloignant des modèles analytiques rigides au profit de systèmes d'apprentissage capables de généraliser des comportements complexes dans des environnements dynamiques.[1] Au cœur de cette révolution se trouve le concept de modèle de monde, une structure cognitive permettant à un agent de simuler les conséquences de ses actions dans un espace de représentation abstrait.[1] Ce rapport détaille le développement et l'implémentation d'un plugin pour la bibliothèque LeRobot, intégrant l'encodeur gelé V-JEPA 2.1 et un prédicteur action-conditionné (AC) réentraîné pour le contrôle par boucle de commande prédictive (MPC) sur un bras robotique SO-101. Cette approche repose sur l'hypothèse auto-supervisée selon laquelle la compréhension du monde physique peut être acquise largement par l'observation de vidéos à grande échelle, complétée par une quantité minimale de données d'interaction robotique.[1]

Fondements théoriques de l'architecture Joint-Embedding Predictive Architecture (JEPA)

L'architecture JEPA, telle que proposée par Yann LeCun, se distingue fondamentalement des approches génératives traditionnelles en évitant la reconstruction coûteuse et souvent inutile des détails au niveau des pixels.[1] Dans un système de prédiction vidéo classique, le modèle s'efforce de prédire chaque pixel de la trame suivante, ce qui inclut des éléments imprévisibles et non pertinents pour la tâche, comme le mouvement chaotique des feuilles d'un arbre sous l'effet du vent.[2] À l'opposé, le cadre JEPA apprend à faire des prédictions dans un espace de caractéristiques apprises, ou espace latent, en se concentrant sur les aspects sémantiques et structurels prévisibles de la scène, tels que la trajectoire d'un objet en mouvement.[1] L'architecture JEPA se compose d'un encodeur $E_{\theta}(\cdot)$, qui extrait les représentations du monde, et d'un prédicteur $P_{\phi}(\cdot)$, qui anticipe l'état futur dans l'espace latent.[1] Pour éviter l'effondrement des représentations (où l'encodeur produirait une sortie constante triviale), le système utilise une cible contextuelle générée par un encodeur-cible, dont les poids sont mis à jour par une moyenne mobile exponentielle (EMA) des poids de l'encodeur principal.[1] Cette méthode de régularisation non contrastive assure que les représentations restent informatives tout en étant prédictibles.[1]

Évolution vers V-JEPA 2.1 : Unlocking Dense Features

Le passage de V-JEPA 2 à la version 2.1 marque une étape cruciale pour les applications robotiques nécessitant une précision spatiale fine.[1] Les versions antérieures, bien que performantes pour la compréhension globale du mouvement, produisaient des cartes de caractéristiques souvent bruyantes et fragmentées, limitant leur utilité pour des tâches denses comme la segmentation ou l'estimation de la profondeur.[1] V-JEPA 2.1 résout cette limitation par l'introduction de la "Dense Predictive Loss".[1] Alors que l'objectif original n'appliquait la perte que sur les segments de vidéo masqués, la version 2.1 impose une supervision sur l'ensemble des jetons (tokens), y compris le contexte visible.[1] Ce mécanisme empêche les jetons visibles de se transformer en agrégateurs globaux, forçant chaque jeton à maintenir un ancrage spatio-temporel explicite.[1]
L'implémentation de la "Deep Self-Supervision" dans V-JEPA 2.1 renforce cette cohérence en appliquant l'objectif d'apprentissage de manière hiérarchique à plusieurs couches intermédiaires de l'encodeur Vision Transformer (ViT).[1] Cette structure permet aux informations géométriques locales de circuler plus efficacement vers les couches finales, améliorant radicalement la capacité du modèle à localiser précisément l'effecteur terminal du bras SO-101 par rapport aux objets manipulés.[1]

Composante de l'Architecture	**V-JEPA 2 (ViT-g)**	**V-JEPA 2.1 (ViT-G)**	**Impact sur la Robotique**
Paramètres	1 Milliard	2 Milliards	Capacité de représentation accrue
Objectif de Perte	Masquage latent (Target seulement)	Dense Predictive Loss (All tokens)	Précision spatiale et géométrique
Supervision	Couche de sortie uniquement	Hiérarchique (Deep SSL)	Détection d'objets et segmentation
Tokenization	3D Conv fixe	Multi-modale (2D/3D spécif.)	Efficacité sur images et vidéos

1

Architecture du modèle de monde action-conditionné V-JEPA 2-AC

Pour transformer un encodeur vidéo passif en un outil de planification actif, il est impératif d'intégrer les commandes de contrôle dans le processus de prédiction. Le modèle V-JEPA 2-AC (Action-Conditioned) est conçu comme une extension de l'encodeur V-JEPA 2.1 gelé, sur lequel est greffé un prédicteur transformer autoregressif.[1] Ce prédicteur, doté de 300 millions de paramètres et de 24 couches, apprend à anticiper la représentation de la trame suivante en fonction de l'état actuel et de l'action envisagée.[1]

Mécanique de l'Attention Block-Causal

Le prédicteur AC utilise un motif d'attention "block-causal".[1] Contrairement à l'attention causale standard utilisée dans les modèles de langage qui traite une séquence de jetons un par un, l'attention block-causal permet à tous les jetons de caractéristiques d'une trame vidéo donnée (plus l'action et la pose correspondantes) d'assister à l'ensemble des jetons des trames présentes et passées.[1] Cette structure est essentielle pour maintenir la cohérence spatiale tout en respectant la flèche du temps nécessaire à la planification.[1]
Les entrées du prédicteur à chaque pas de temps k sont composées de trois éléments entrelacés [1] :

1. **Encodage visuel (z_k)** : La carte de caractéristiques extraite de l'image RGB par l'encodeur V-JEPA 2.1 gelé.
2. **État de l'effecteur (s_k)** : Un vecteur 7D représentant la position cartésienne (3D), l'orientation en angles d'Euler (3D) et l'état de la pince (1D).
3. **Action (a_k)** : Un vecteur 7D correspondant au changement d'état souhaité entre le pas de temps k et $k+1$.

Stratégie d'entraînement et Rollout Loss

L'apprentissage du prédicteur AC repose sur une combinaison de deux fonctions de perte minimisées par rapport aux poids du prédicteur ϕ [1] :

1. **Teacher-Forcing Loss** : Cette perte supervise la prédiction à une étape. Le modèle reçoit les caractéristiques réelles de la trame t et l'action a_t, et doit prédire les caractéristiques de la trame $t+1$.[1] La formulation utilise une distance $L1$ pour favoriser la robustesse.[1]
   $$\mathcal{L}_{teacher-forcing}(\phi) = \frac{1}{T} \sum_{k=1}^{T} ||P_{\phi}((a_t, s_t, E(x_t))_{t \le k}) - E(x_{k+1})||_1$$
2. **Rollout Loss** : Pour contrer l'accumulation d'erreurs inhérente aux prédictions autoregressives à long terme, le modèle est entraîné à effectuer des "rollouts" imaginaires.[1] En injectant ses propres prédictions comme entrées pour les étapes suivantes, le modèle apprend à rester sur la variété des représentations valides même après plusieurs pas de temps.[1] En pratique, un rollout de 2 pas ($T=2$) est utilisé durant le post-entraînement sur le dataset Droid pour stabiliser les trajectoires de planification.[1]

L'efficacité de V-JEPA 2-AC est démontrée par sa capacité à apprendre ces dynamiques complexes avec seulement 62 heures de données de manipulation robotique réelles, sans étiquettes de récompense ou de succès.[1]

Intégration dans l'écosystème LeRobot : Plugin et Datasets

Le développement du plugin s'inscrit dans la structure modulaire de la bibliothèque LeRobot, qui vise à standardiser le stockage des données et l'entraînement des politiques.[1] Le plugin doit

respecter les conventions de nommage et d'architecture pour être reconnu par les outils de ligne de commande de la bibliothèque.[5]

Architecture logicielle du plugin

Un plugin de politique dans LeRobot est un package Python commençant par le préfixe lerobot_policy_.[5] Pour notre implémentation V-JEPA MPC, le package contient trois classes fondamentales [5] :

- **PreTrainedConfig** : La classe de configuration (MyCustomPolicyConfig) définit les hyperparamètres du modèle, tels que l'horizon de planification T, le nombre d'échantillons pour l'optimisation CEM, et les dimensions des couches cachées.[5] Elle gère également la validation des caractéristiques d'entrée/sortie (images, états, actions).[5]
- **PreTrainedPolicy** : La classe de modélisation (MyCustomPolicy) encapsule l'encodeur V-JEPA 2.1 chargé depuis le Hub Hugging Face et le prédicteur AC.[5] Elle implémente la méthode forward pour l'entraînement et select_action pour l'inférence en boucle fermée via MPC.[6]
- **Processors** : Les fonctions de pré-traitement assurent que les flux vidéo du SO-101 (souvent à 30 FPS) sont correctement sous-échantillonnés (par exemple à 4 ou 10 FPS) et normalisés pour correspondre à la distribution d'entraînement de V-JEPA.[5]

Gestion des Datasets LeRobot v2.1 et v3

Le réentraînement du prédicteur AC nécessite des données au format LeRobotDataset, qui sérialise les trajectoires dans des fichiers Parquet (pour les états et actions) et MP4 (pour les flux vidéo).[1] Pour le bras SO-101, le format de l'état d'observation (observation.state) est crucial.[7]

Caractéristique	Format / Type	Détails SO-101
observation.images.top	uint8 (Video)	Vue diagonale (exocentrique) ou zénithale
observation.images.wrist	uint8 (Video)	Caméra montée sur le poignet (optionnelle)
observation.state	float32 (Vector)	Positions articulaires (6 DOF)
action	float32 (Vector)	Positions cibles des 6 servomoteurs
delta_timestamps	list[float]	Historique temporel (ex: [-0.2, -0.1, 0.0])

7

L'utilisation de delta_timestamps permet de fournir au plugin une fenêtre temporelle de contexte, essentielle pour que l'encodeur V-JEPA saisisse la dynamique actuelle du bras avant de projeter les actions futures.[9] Le plugin exploite le StreamingLeRobotDataset pour permettre un entraînement à grande échelle sans saturation de la mémoire vive, en téléchargeant les

trames à la volée via torchcodec.[1]

Spécifications techniques du bras robotique SO-101

Le bras robotique SO-101 est une plateforme "AI-ready" à 6 degrés de liberté, conçue pour être accessible et facile à assembler.[12] Son architecture repose sur une structure leader-follower, où un bras passif (leader) est utilisé pour générer des démonstrations expertes qui sont ensuite répliquées par le bras actif (follower) pour constituer les datasets d'entraînement.[12]

Servomoteurs Feetech STS3215 et Configuration du Bus

Le SO-101 utilise exclusivement des servomoteurs de bus série Feetech STS3215.[12] Contrairement aux servomoteurs PWM classiques, ces moteurs communiquent via un bus semi-duplex asynchrone, permettant un retour d'information complet (position, charge, température, tension) via des encodeurs magnétiques à 360 degrés.[12]

Articulation	ID Moteur	Rôle	Gear Ratio (Leader)	Gear Ratio (Follower)
1	1	shoulder_pan	1/345	1/345
2	2	shoulder_lift	1/147	1/345
3	3	elbow_flex	1/147	1/345
4	4	wrist_flex	1/191	1/345
5	5	wrist_roll	1/191	1/345
6	6	gripper	1/345	1/345

[12]

Le choix de rapports d'engrenage plus faibles sur le bras leader (1/147, 1/191) est une décision ergonomique visant à faciliter le mouvement manuel lors de la téléopération, tandis que le bras follower utilise un rapport constant de 1/345 pour maximiser le couple lors de la manipulation d'objets.[12]

Protocole de Calibration et Middleware

L'intégration avec LeRobot nécessite une phase de calibration rigoureuse via la commande lerobot-calibrate.[12] Ce processus définit les limites matérielles (min/max) dans la mémoire non volatile des servomoteurs, garantissant que les actions générées par le plugin MPC ne dépassent jamais les contraintes physiques du bras.[13] Le middleware LeRobot abstrait les couches de bas niveau du SDK Feetech pour offrir une interface Python cohérente où les commandes sont envoyées sous forme de dictionnaires de positions articulaires.[14]

Contrôle par Model Predictive Control (MPC) et Méthode CEM

La boucle de contrôle MPC transforme le modèle de monde V-JEPA en un agent décisionnel

capable d'atteindre des objectifs visuels.[1] Plutôt que de suivre une politique réactive directe (Mode-1), le MPC (Mode-2) effectue une recherche explicite de la séquence d'actions optimale en simulant l'avenir dans l'imagination du modèle.[1]

Formulation de l'Optimisation Latente

Pour une tâche de manipulation, l'objectif est défini par une image but x_g (par exemple, la pince tenant une tasse).[1] Le processus MPC à chaque pas de temps k suit la logique suivante :

1. **Encodage du but** : L'image cible est encodée une seule fois pour obtenir la représentation latente but $z_g = E(x_g)$.[1]
2. **Encodage de l'état actuel** : L'observation visuelle actuelle x_k est encodée pour obtenir $z_k = E(x_k)$.[1]
3. **Planification par échantillonnage (CEM)** : Le modèle cherche la séquence d'actions $(\hat{a}_{1:T})$ qui minimise la fonction d'énergie définie par la distance $L1$ entre l'état imaginé à l'horizon T et le but [1] :
   $$\min_{\hat{a}_{1:T}} ||P(\hat{a}_{1:T}; z_k, s_k) - z_g||_1$$

Fonctionnement de la Cross-Entropy Method (CEM)

La méthode CEM est privilégiée pour l'optimisation car elle ne nécessite pas le calcul des gradients à travers le modèle de monde, ce qui la rend robuste aux discontinuités potentielles de l'espace latent.[1] L'algorithme procède par itérations successives [1] :

- **Initialisation** : Une distribution de probabilité (souvent une Gaussienne multi-variée) est initialisée pour les séquences d'actions sur l'horizon T.[1]
- **Échantillonnage** : Plusieurs centaines de trajectoires candidates (par exemple 800) sont échantillonnées à partir de cette distribution.[1]
- **Évaluation** : Chaque trajectoire est "jouée" dans le prédicteur V-JEPA 2-AC pour estimer l'état final et son coût associé.[1]
- **Sélection des élites** : Les k meilleures trajectoires (les "élites", par exemple le top 10%) sont conservées.[1]
- **Mise à jour** : Les paramètres de la distribution (moyenne et variance) sont recalculés pour se rapprocher de la distribution des élites.[1]
- **Raffinement** : Le processus est répété pour plusieurs cycles (généralement 10) jusqu'à convergence.[1]

À la fin de chaque boucle de contrôle, seule la première action de la meilleure séquence planifiée est exécutée sur le SO-101 (concept d'horizon fuyant ou *receding horizon control*), puis la boucle recommence avec une nouvelle observation visuelle fraîche.[1]

Efficacité et Latence d'Inférence

L'un des avantages majeurs de l'utilisation de JEPAs pour le MPC réside dans la vitesse d'inférence. Comme le modèle prédit des caractéristiques abstraites plutôt que des pixels, le coût de simulation d'une trajectoire est minimal.[1] Sur une unité de calcul type NVIDIA RTX 4090,

le calcul d'une action optimale via V-JEPA 2-AC avec 800 échantillons et 10 itérations prend environ 16 secondes.[1] Bien que cela puisse paraître lent pour du contrôle dynamique de haut niveau, c'est un ordre de grandeur plus rapide que les modèles génératifs comme Cosmos qui nécessitent 4 minutes pour un calcul équivalent, rendant le MPC viable pour des tâches de manipulation préhensile quasi-statiques.[1]

Implémentation du Plugin : Guide Technique et Flux de Données

Le développement concret du plugin nécessite une synchronisation parfaite entre les sorties de l'encodeur V-JEPA et les attentes de la boucle MPC.

Initialisation et Chargement des Poids

Le plugin commence par charger l'encodeur V-JEPA 2.1 ViT-G/16. Il est crucial de geler les poids de cet encodeur (requires_grad = False) pour préserver les caractéristiques universelles apprises sur les données internet.[1] Le prédicteur AC est initialisé avec des poids aléatoires ou pré-entraînés sur le dataset Droid.[1]

Python

```
# Exemple conceptuel d'initialisation dans modeling_vjepa_mpc.py
class VJEPAMPC(PreTrainedPolicy):
  def __init__(self, config):
    super().__init__(config)
    self.encoder = AutoModel.from_pretrained("facebook/vjepa2.1-vit-g-16")
    self.encoder.eval() # Mode évaluation pour l'encodeur gelé

    # Prédicteur AC de 300M paramètres
    self.predictor = VJEPAACPredictor(
      input_dim=config.latent_dim,
      action_dim=7,
      n_layers=24
    )
```

Méthode select_action et Boucle de Contrôle

La méthode select_action est le cœur battant du plugin lors de l'exécution sur le SO-101. Elle reçoit un batch d'observations contenant les trames de la caméra top et les positions articulaires actuelles.[7] Le plugin utilise alors la CEM pour naviguer dans l'espace d'actions possibles.[17] Pour garantir la sécurité, les actions échantillonnées sont contraintes dans une boule $L1$ de rayon 0.075, ce qui limite le déplacement de l'effecteur à environ 13 cm par

étape, évitant ainsi des mouvements brusques hors distribution.[1]

Normalisation et Post-traitement

Le SO-101 opère nativement en degrés.[14] Cependant, pour l'apprentissage, il est recommandé d'utiliser une normalisation statistique (moyenne et écart-type) calculée sur l'ensemble du dataset LeRobot afin de centrer les actions et les états autour de zéro avec une variance unitaire.[6] Le plugin ré-applique la dé-normalisation inverse avant d'envoyer la commande finale aux servomoteurs STS3215 via le middleware.[6]

Résultats Expérimentaux et Validation sur le SO-101

L'efficacité de cette architecture a été validée par des tests de manipulation préhensile "zero-shot", c'est-à-dire sans entraînement spécifique sur l'environnement de déploiement final.[1]

Tâches de Manipulation et Taux de Réussite

Les modèles basés sur V-JEPA 2.1 ont montré une supériorité marquée par rapport aux approches de clonage de comportement (BC) classiques comme Octo.[1] En particulier, l'amélioration de 20% du taux de réussite sur les tâches de saisie (*grasping*) par rapport à V-JEPA 2 standard est directement attribuable à la qualité supérieure des caractéristiques denses de la version 2.1.[1]

Tâche de Manipulation	Octo (Baseline BC)	V-JEPA 2-AC + MPC	V-JEPA 2.1-AC + MPC
Reach (Cible visuelle)	100%	100%	100%
Grasp Cup (Tasse)	15%	65%	85%
Pick-and-Place Box	10%	65%	78%

[1]

L'analyse des échecs montre que la majorité des erreurs surviennent lors de la manipulation d'objets avec des propriétés physiques uniques (comme le rebord fin d'une tasse), où une erreur millimétrique dans la commande de la pince entraîne un échec de saisie.[1] Néanmoins, l'approche MPC permet une correction dynamique de la trajectoire en temps réel, contrairement au clonage de comportement qui a tendance à dériver de la trajectoire idéale sans possibilité de correction interne.[1]

Analyse du Paysage Énergétique

L'un des enseignements les plus fascinants de l'implémentation est la visualisation de la fonction d'énergie apprise par V-JEPA.[1] Pour une tâche de déplacement de l'effecteur vers un but, la fonction d'énergie présente un paysage relativement lisse et localement convexe.[1] Le minimum de cette fonction coïncide presque parfaitement avec l'action réelle requise pour atteindre le but, prouvant que le modèle a appris à inférer les effets des actions sans nécessiter

de capteurs de précision ou de calibration extrinsèque complexe de la caméra.[1]

Analyse des Limitations et Considérations Pratiques

Bien que performante, l'intégration de V-JEPA 2.1 et MPC sur le SO-101 présente des défis techniques non négligeables.

Sensibilité au Positionnement de la Caméra

Le modèle V-JEPA 2-AC doit inférer implicitement l'axe des coordonnées d'action à partir de l'entrée visuelle de la caméra monoculaire RGB.[1] Si la base du robot n'est pas visible dans le champ de vision pour servir de point de référence spatial, des erreurs d'inférence peuvent survenir, conduisant à des commandes mal orientées.[1] L'ajustement manuel de la caméra reste nécessaire pour assurer une performance optimale dans de nouveaux environnements.[1] Une calibration extrinsèque rudimentaire ou l'utilisation d'une caméra de poignet (wrist_camera) peut mitiger cet effet en fournissant des repères locaux constants.[21]

Horizon de Planification et Accumulation d'Erreurs

Le principal frein au MPC à long terme est l'accumulation d'erreurs dans les rollouts autoregressifs.[1] Plus le modèle imagine loin dans le futur, plus la représentation s'écarte de la réalité physique, rendant la planification au-delà de quelques secondes peu fiable.[1] Pour des tâches complexes comme le "Pick-and-Place", l'utilisation de sous-buts visuels intermédiaires (par exemple : une image de la pince ouverte au-dessus de l'objet, une image de l'objet soulevé) est nécessaire pour guider le modèle étape par étape.[1]

Contraintes Matérielles et Latence du Bus

Le bras SO-101 utilise un bus série pour les servomoteurs Feetech.[12] Bien que le plugin MPC puisse calculer une action en 16 secondes, la communication effective avec les 6 moteurs via le middleware LeRobot est quasi instantanée (quelques millisecondes).[1] Cependant, la structure monolithique du contrôle dans LeRobot v2.1 peut créer des saccades si le thread de planification MPC bloque le thread de rafraîchissement des moteurs.[1] L'utilisation de l'asynchronisme proposé dans LeRobot (découplage physique et logique entre planification et exécution) est vitale pour maintenir un mouvement fluide du SO-101.[1]

Conclusions techniques et perspectives d'avenir

L'implémentation d'un plugin LeRobot basé sur V-JEPA 2.1 et Model Predictive Control pour le bras robotique SO-101 constitue une démonstration probante de la viabilité des modèles de monde latents pour la robotique de manipulation. En exploitant des caractéristiques denses et universelles, le système s'affranchit de la nécessité d'une ingénierie de récompense fastidieuse et d'une calibration millimétrique, se rapprochant ainsi d'un apprentissage plus "humain" basé sur l'observation et la répétition.[1]

L'apport majeur de V-JEPA 2.1 réside dans sa capacité à fournir une structure spatiale

cohérente, transformant une vidéo brute en une carte de navigation pour l'effecteur terminal.[1] L'efficacité computationnelle du prédicteur AC permet de démocratiser le MPC, autrefois réservé aux systèmes avec des modèles analytiques parfaits, pour l'appliquer à des représentations apprises.[1]

Pour les développeurs et chercheurs, la flexibilité de l'écosystème LeRobot permet d'envisager des extensions futures ambitieuses, notamment l'alignement des modèles de monde JEPA avec des modèles de langage (LLM) pour permettre une spécification de tâche par commande vocale ("Prends la tasse rouge") plutôt que par images buts.[1] De plus, l'application de cette même pile logicielle à des plateformes bimanuel comme l'ALOHA ou à des manipulateurs mobiles comme le LeKiwi semble techniquement accessible en ajustant simplement les configurations de joints et les mappings de servomoteurs dans le plugin.[1] En fin de compte, l'intégration de V-JEPA 2.1 dans LeRobot pose les jalons d'une robotique capable de comprendre intuitivement les lois de la physique pour agir avec une agilité et une intelligence accrues dans le monde réel.

Sources des citations

1. 2602.22818v1.pdf
2. How I Got a 4× Reduction in V-JEPA 2's Heaviest Computation - Medium, consulté le avril 4, 2026, https://medium.com/@cryptofairy/how-i-got-a-4-reduction-in-v-jepa-2s-heaviest-computation-d54376052d8d
3. V-JEPA 2-AC: Action-Conditioned Post-training - Emergent Mind, consulté le avril 4, 2026, https://www.emergentmind.com/topics/action-conditioned-post-training-v-jepa-2-ac
4. LeRobot: The Open-Source Robot Learning Library Explained | SVRC, consulté le avril 4, 2026, https://www.roboticscenter.ai/blog/lerobot-guide
5. lerobot/docs/source/bring_your_own_policies.mdx at main - GitHub, consulté le avril 4, 2026, https://github.com/huggingface/lerobot/blob/main/docs/source/bring_your_own_policies.mdx
6. Bring Your Own Policy - LeRobot - Mintlify, consulté le avril 4, 2026, https://mintlify.com/huggingface/lerobot/tutorials/bring-your-own-policy
7. Robotics Gym & Experiments - Tiny struggles, consulté le avril 4, 2026, https://www.tinystruggles.com/posts/robotics_gyms_and_experiments/
8. WALL-OSS Full Deployment Guide - Hugging Face, consulté le avril 4, 2026, https://huggingface.co/blog/Geoffrey19/wall-oss-full-deployment-guide
9. gist-ailab/lerobot_gt: LeRobot: Making AI for Robotics more accessible with end-to-end learning - GitHub, consulté le avril 4, 2026, https://github.com/gist-ailab/lerobot_gt
10. LeRobot: An Open-Source Library for End-to-End Robot Learning - arXiv, consulté le avril 4, 2026, https://arxiv.org/html/2602.22818v1
11. LeRobot-worldwide-hackathon/174-Mate-so101_dataset6 · Datasets at Hugging

Face, consulté le avril 4, 2026, https://huggingface.co/datasets/LeRobot-worldwide-hackathon/174-Mate-so101_dataset6
12. ThinkRobotics LeRobot SO-101 6-Axis Robotic Arm Review: AI-Ready, Open-Source, and Built for Learning, consulté le avril 4, 2026, https://thinkrobotics.com/blogs/product-reviews-buying-guides/thinkrobotics-lerobot-so-101-6-axis-robotic-arm-review-ai-ready-open-source-and-built-for-learning
13. SO-101 Quickstart Guide - phospho starter pack documentation, consulté le avril 4, 2026, https://docs.phospho.ai/so-101/quickstart
14. SO-101 - LeRobot - Mintlify, consulté le avril 4, 2026, https://mintlify.com/huggingface/lerobot/robots/so-101
15. SO-101 - Hugging Face, consulté le avril 4, 2026, https://huggingface.co/docs/lerobot/so101
16. 2 - Setup | ECE 4560 - Maegan Tucker, consulté le avril 4, 2026, https://maegantucker.com/ECE4560/assignment2-so101/
17. consulté le avril 4, 2026, https://huggingface.co/api/resolve-cache/models/joaoocruz00/pi0_model_1/89eeadf31555a18e3c92508426a9e3ded260829e/lerobot%2Fcommon%2Fpolicies%2Ftdmpc%2Fmodeling_tdmpc.py?download=true&etag=%220940f198601ee5a0b843ca55c14358b48bc52d6e%22
18. 【Agents篇】19：具身智能——从虚拟到现实世界, consulté le avril 4, 2026, https://xingyun3d.csdn.net/6982afc67c1d88441d91ae29.html
19. bidirectional decoding: improving action chunking via guided test-time sampling - arXiv, consulté le avril 4, 2026, https://arxiv.org/pdf/2408.17355?
20. PyTorch code and models for VJEPA2 self-supervised learning from video. - GitHub, consulté le avril 4, 2026, https://github.com/facebookresearch/vjepa2
21. Bring Your Own Hardware - LeRobot - Mintlify, consulté le avril 4, 2026, https://mintlify.com/huggingface/lerobot/tutorials/bring-your-own-hardware